

Nirnay

On-Chain Policy Decision Auditor | Complete Project Plan | RAG + LLM + Blockchain Audit Trail System

1. Problem Statement

Institutions — colleges, startups, compliance teams — make policy-based decisions constantly (scholarship eligibility, vendor compliance, regulatory checks), but the process is slow (manual document search) and unauditable (decisions live in emails or memory with no tamper-proof record). Existing solutions (Polygraf, OriginStamp) solve this only for large enterprises using private, vendor-controlled logging. Small institutions have no affordable, independently verifiable option.

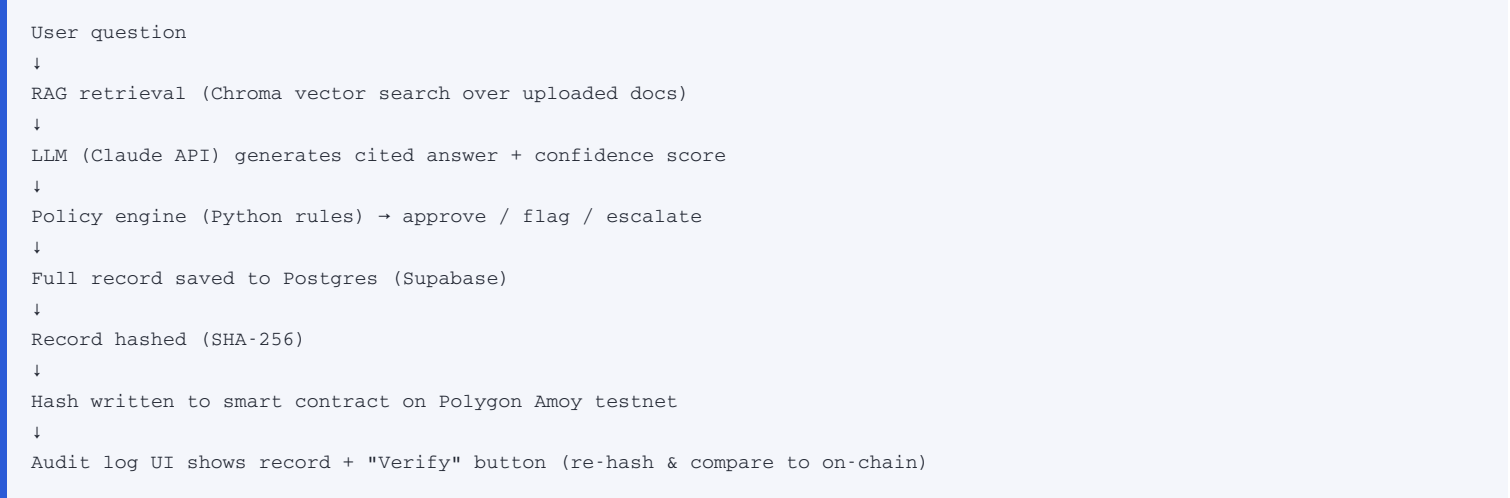
2. Proposed Solution

An AI-powered Q&A system that answers policy questions from an institution's own documents with cited evidence, routes each answer through a lightweight policy engine (approve / flag / escalate), and permanently hashes every decision record onto a public blockchain — making the audit trail independently verifiable by anyone, not just trusted on a vendor's word.

3. Target Audience

Tier	Audience	Use Case
Primary	Colleges & universities	Scholarship eligibility, admissions, academic integrity, disciplinary decisions
Secondary	Startup compliance / legal teams	Vendor contract review, data-handling policy checks
Tertiary	Internal audit / risk teams	Proving due diligence in policy-based decisions

4. End-to-End Architecture



5. Tech Stack

Layer	Tool
Frontend	Stitch / React
Backend	FastAPI
Database	Supabase (Postgres)
Vector store	Chroma
RAG framework	LlamaIndex or LangChain
Embeddings	OpenAI or sentence-transformers
LLM	Claude API
Policy engine	Plain Python (rule-based)
Smart contract	Solidity
Blockchain network	Polygon Amoy testnet

Chain integration	web3.py
Hashing	hashlib (SHA-256)

6. Component Build Checklist

A. Document Ingestion

- ☐ Upload endpoint (PDF/text) in FastAPI
- ☐ Text extraction (PyPDF2 or similar)
- ☐ Chunking logic (e.g. 500-token chunks with overlap)
- ☐ Embedding generation for each chunk
- ☐ Store embeddings in Chroma with metadata (source doc, page, upload date)

B. RAG Retrieval + Q&A

- ☐ Question endpoint: takes user question
- ☐ Retrieve top-k relevant chunks from Chroma
- ☐ Construct prompt: question + chunks + instruction to cite sources and self-rate confidence
- ☐ Call Claude API, parse response (answer + citations + confidence)

C. Policy Engine

- ☐ Define rules: confidence threshold for auto-approve
- ☐ Keyword/flag list for content requiring escalation (e.g. "data sharing," "third-party," "termination")
- ☐ Decision output: approved / flagged / escalated
- ☐ (Optional) simple reviewer dashboard for flagged items to be manually resolved

D. Database Layer

- ☐ Supabase schema: **assessments** table — id, question, answer, sources, confidence, decision, reviewer_id, timestamp, record_hash
- ☐ **institutions** table (reuse Pramaan's trust-tier concept for the shared narrative)
- ☐ Write endpoint to persist each assessment

E. Hashing

- ☐ Function to serialize assessment record deterministically (consistent field order — critical for reproducible hashing)
- ☐ SHA-256 hash of the serialized record
- ☐ Store hash alongside record in Postgres

F. Blockchain Layer

- ☐ Solidity contract: mapping recordId → hash, plus timestamp and submitter address
- ☐ Deploy to Polygon Amoy testnet
- ☐ web3.py integration in FastAPI to call contract's write function
- ☐ Read/verify function: given a record, re-hash it and compare to on-chain value
- ☐ Reuse Pramaan's existing contract deployment + web3.py wiring wherever possible

G. Frontend

- ☐ Upload page (documents)
- ☐ Chat/Q&A page (ask question, see cited answer + confidence + decision)
- ☐ Audit log page (list of past assessments, each with a "Verify on-chain" button)
- ☐ Verify button triggers re-hash + comparison, shows green/red result

H. Demo Data

- ☐ Prepare 3–5 realistic sample policy documents (e.g. a fake scholarship policy, a fake vendor data policy)
- ☐ Prepare 5–6 realistic demo questions to ask live, including at least one that should get flagged/escalated

7. Suggested Build Order

Phase	What	Why This Order
1	Document ingestion + Chroma setup	Foundation everything else depends on

2	RAG retrieval + Claude Q&A (no policy engine yet)	Get the "wow" demo piece working first
3	Policy engine (simple rules)	Fast to add once Q&A works
4	Postgres schema + save records	Needed before hashing makes sense
5	Hashing function	Small, isolated, low-risk
6	Blockchain contract + web3.py write	Highest reuse from Pramaan — do once the above is stable
7	Frontend polish + audit log + verify button	This is what you'll click through live for judges
8	Demo data + rehearsal	A clean demo matters more than extra features

8. Competitive Landscape & Differentiation

Component	Already Exists	Your Difference
RAG over policy documents	Yes – standard in most enterprise AI-search tools	Not a differentiator on its own
Cited answers + confidence scoring	Yes – common baseline now	Expected, not novel
Logging every AI/RAG decision with attribution	Yes – Polygraf AI does this	Polygraf is closed, on-prem/enterprise only
Hash-chained tamper-evidence	Yes – Polygraf uses prev_hash/record_hash chains	Private hash-chains are vendor-trust-based, not independently verifiable
Blockchain-anchored AI decision trails	Yes – OriginStamp/OriginVault's core product	Closest existing match – must differentiate explicitly
Permissioned blockchain for audit trails	Yes – academic research (healthcare, gov't)	Research-stage only, not shipped products
Public testnet blockchain (not private)	Rare	Key differentiator – verifiable by anyone, not just the vendor
Targeting small/under-resourced institutions	No	Key differentiator – existing players target large enterprises/govt
Unified trust platform (credentials + decisions)	No	Most original angle – extends Pramaan's trust-tier model from records to decisions

Pitch line: Enterprise tools like Polygraf and OriginStamp solve this for large regulated companies using private, vendor-controlled logging. We bring the same tamper-evident trust model — public, independently verifiable, and affordable — to institutions that can't afford enterprise compliance software, as a natural extension of our existing credential-verification trust system (Pramaan).

9. Demo Script

1. Show uploaded policy documents.
2. Ask a straightforward question → get instant cited answer, auto-approved.
3. Ask a question designed to trigger escalation → show the policy engine flagging it.
4. Open the audit log → click "Verify" on a past record → show it matches the on-chain hash.
5. *(Strongest beat)* Manually edit the database record directly, then click "Verify" again → show the mismatch/red flag. This is the moment that makes "tamper-evident" concrete instead of abstract.